# Monicks PRD — VDOC 2.0 Multi-Action Commands

Oct 22, 2026 · @monick

## Overview

The prototype lets VDOC run several edits from one spoken Tanglish sentence, in the right order, on the right lines.

**Problem.** People say more than one edit at once. Old VDOC picks one intent per command and drops the rest.

**Example.** "second line delete pannu, apram second line bold pannu"

- Old VDOC: deletes line 2, then stops. The bold is lost.
- Expected: delete line 2, then bold the new line 2 (the line that moved up).

**Why not a chat AI.** In the VDOC paper, a Groq LLM got 0.8% on 500 Tanglish commands; DocEditBERT got 100%. This prototype keeps that small, offline model and extends it.

**Owner:** Monick. **Teammate part:** follow-up commands with memory (see Vickys PRD).

## Goals and non-goals

The prototype must run 2–3 actions from one command, in English and Tanglish, fully offline.

**Goals**

1. Detect where each action starts inside one command.
2. Run the actions in the order spoken.
3. Give a missing target to an action by borrowing it from the action before.
4. Update line positions after every edit, so later actions hit the right line.
5. Keep single-action commands working as well as old VDOC.
6. Run offline on a laptop, with the same React + Flask app.

**Non-goals (for this prototype)**

- Follow-up commands across sentences ("adha", "innum", undo). That is Vicky's part.
- Fixing speech-to-text errors beyond what old VDOC already does.
- More than 3 actions in one command.
- New action types beyond the 10 VDOC already supports.

## Hard cases

Four cases decide if the prototype works. Each must pass in both English and Tanglish.

| Case | Example command | Old VDOC does | Prototype must do |
|---|---|---|---|
| Two actions, two targets | "first line bold pannu, apram third line delete pannu" | Only the first action | bold(line 1), then delete(line 3) |
| One target, two actions | "first line bold-um italic-um pannu" | Only bold, or confused | bold(line 1), italic(line 1) |
| Lines shift after delete | "second line delete pannu, apram second line bold pannu" | Deletes, then stops | delete(line 2), then bold the new line 2 |
| Different joining words | "apram", "aprom", "pannitu", "-um", "and", "then" | Not recognised | All split correctly |

Open question: should "delete pannitu bold pannu" on the **same** line be rejected? (The line no longer exists after delete.) Proposed answer: show a short message and skip the second action.

## Solution design

Three steps run after speech-to-text and before the editor: split, share, re-check.

**Step 1 — Split.** Extend DocEditBERT's token labels with an action-boundary tag. Each word gets tagged as the start of an action, inside an action, or a joining word.

```
second  line  delete  pannu  apram  second  line  bold  pannu
B-ACT   I-ACT I-ACT   I-ACT  JOIN   B-ACT   I-ACT I-ACT I-ACT
```

Result: two chunks, each sent to the existing intent + slot model.

**Step 2 — Share.** If a chunk has an action but no target, copy the target from the chunk before it.

```
"first line bold-um italic-um pannu"
chunk 1: bold,   target = line 1
chunk 2: italic, target = (none) -> borrow line 1
```

**Step 3 — Re-check positions.** Keep a list of line ids. After each edit, rebuild the list, then resolve the next action's line number on the new list.

```
before:          [1 Title, 2 Intro, 3 Budget]
delete(line 2):  [1 Title, 2 Budget]
bold(line 2):    -> Budget
```

**Model rule.** Transformer only (DistilBERT, as in VDOC). No CNN, SVM, LSTM or RNN, per the guide's rule.

## Dataset: DocEdit-Compound

Target size: about 1,000 multi-action commands, half English and half Tanglish, built from the existing DocEdit-1000.

### How to build it

1. Take two or three single commands from DocEdit-1000.
2. Join them with a joining word ("apram", "pannitu", "-um", "and", "then").
3. Save the label for each action: action type, target, order.
4. Hand-check a random 100 for natural wording. Fix the odd ones.
5. Hand-write about 100 extra commands that only make sense together (the shift case, the shared-target case).

### One row looks like this

```
command,lang,n_actions,actions
"second line delete pannu apram second line bold
pannu",tanglish,2,"delete(2);bold(2*)"
```

`2*` = line 2 after the delete is applied.

**Split:** 70% train, 15% validation, 15% test. The hand-written 100 go only into test.

## Architecture and code changes

The old VDOC app stays the base: React/Vite frontend, Flask backend, DocEditBERT. Only the middle of the pipeline changes.

### Pipeline (new parts in bold)

1. Mic → speech-to-text (unchanged)
2. **Splitter** → cuts the text into action chunks
3. Intent + slot model on each chunk (existing DocEditBERT)
4. **Target sharer** → fills missing targets
5. **Position tracker** → re-maps line numbers after each edit

6. Editor applies the actions one by one (unchanged executor, called in a loop)

**Code changes**

| Where | Change |
|---|---|
| Backend parse endpoint | Return a **list** of actions instead of one |
| Model training script | Add boundary tags (B-ACT, I-ACT, JOIN) and retrain on DocEdit-Compound |
| New module: splitter | Cut text by boundary tags |
| New module: position tracker | Keep line ids; re-map after each edit |
| Frontend | Show each action as it runs ("1 of 2: delete line 2") |

**Hosting:** runs offline on the laptop first. AWS hosting comes after the prototype works.

## Evaluation and acceptance criteria

The main score is **full-command accuracy**: a command counts only if every action hits the right target, in the right order.

**Compare three systems on the same test set**

| System | What it is |
|---|---|
| Old VDOC | One action per command |
| LLM baseline | A chat LLM given the command and the document |
| VDOC 2.0 (Monick) | Splitter + sharer + position tracker |

**Also report**

- Accuracy per hard case (shared target, line shift, joining words)
- English vs Tanglish, separately
- Time per command on the laptop

**Prototype is done when**

- [ ] All four hard cases work live in the app, in English and Tanglish
- [ ] Single-action accuracy is not lower than old VDOC
- [ ] Test-set numbers are filled for all three systems
- [ ] It runs with no internet

# Milestones and folder setup

Start from a copy of the old VDOC, never the original. Keep one untouched copy as the base.

### Folders

```
vdoc-base/     <- original old VDOC, never edited
vdoc-monick/   <- copy for this PRD
vdoc-vicky/    <- copy for Vicky's PRD
```

Before copying, check the old project still runs on the Mac (frontend + backend + model loads).

### Milestones (in order)

1. Old VDOC runs in `vdoc-monick`
2. Joining-word list + splitter working on typed text
3. DocEdit-Compound built (1,000 commands)
4. DocEditBERT retrained with boundary tags
5. Target sharer + position tracker added
6. All four hard cases work live in the app
7. Test-set numbers for all three systems

Open question: dates for each milestone, to agree with Ms. Sneha George.